

Library Information System Presentation Guide

Everything the team needs to present the project and answer questions about it: what each slide says, what to say out loud, how the code works, and the questions an examiner is likely to ask.

Team

Aryan Raj	24je0594
Ayush Ranjan	24je0596
Banoth Vijender	24je0597
Bathina Shajith	24je0598
Battu Manjula	24je0599
Bhanu Praharsha Kasaraneni	24je0600

Repository: github.com/BhanuPraharsha/SE_lab

Contents

1. How to use this guide	Read first
2. The project in one page	Pitch, 30-second and 2-minute versions
3. Who presents what	Suggested split across six speakers
4. Slide by slide	All 25 slides: purpose, script, facts, questions
5. How the code works	Files, models, every route, the rules
6. Running the app and the live demo	Commands, logins, demo script
7. Question bank	Software engineering theory and project questions
8. Known weak spots (team only)	What an examiner might find, and how to answer
9. Glossary	Terms used in the slides

1. How to use this guide

Everyone should read sections 2, 3 and 4 before the presentation, and at least skim section 7. Section 4 has one block per slide. Each block has four parts:

- **On the slide:** what the audience sees, so you know what you're pointing at.
- **Say this:** a short script in plain words. Don't read it word for word; use it to know what point the slide makes.
- **Facts to know:** the exact details behind the slide, taken from the code. If you're asked "how exactly", the answer is here.
- **Likely questions:** what an examiner might ask about that slide, with an answer.

Section 8 is for the team only. It lists the parts of the code that have gaps, so nobody is caught off guard if an examiner tries something unusual in the demo. Don't bring these up yourselves. If someone asks, answer honestly using the suggested wording.

Every number and rule in this guide comes from `app.py`, `models.py`, `seed.py` and `tests/test_lis.py` in the repository, and the test results come from actually running the app.

2. The project in one page

One-line pitch

LIS is a web app for a college library that handles issuing, returning and reserving books, charges overdue fines automatically, and gives the librarian, the clerk and members their own views.

The 30-second version

A library desk keeps asking three questions: who has this copy, when is it due and what is the fine, and who asked for it first. With a paper register, each one means searching by hand. We built a Flask web app backed by an SQLite database that answers all three. The clerk issues and returns books, and the system enforces loan limits by member category, charges Rs. 2 per late day and holds returned books for the next member in the queue for seven days. The librarian manages members and prints reports. Members search the catalogue and reserve books that are out.

The 2-minute version

Use this if you're asked to "explain the project" without slides.

1. **Problem.** Manual circulation is slow and error-prone. Loan periods depend on the member's category, fines are worked out by hand, and there's no fair queue for books that are out.
2. **Users.** Three roles. The librarian manages members and runs reports. The clerk works the counter: catalogue, issue and return. The member searches and reserves.
3. **Rules.** Undergraduates can borrow 2 books for 30 days, postgraduates 4 for 30, research scholars 6 for 90 and faculty 10 for 180. A late return costs Rs. 2 per day. When a reserved book comes back, it's held for the first member in the queue for 7 days. The unused report lists books with no issue in 5 years.
4. **Design.** A browser talks to one Flask app (`app.py`). SQLAlchemy maps four classes (Member, Book, Issue, Reservation) to an SQLite file. We modelled it with a context DFD, two level-1 DFDs, and use case, class, sequence, activity and state diagrams.
5. **Testing.** Three pytest unit tests (due date, fine, hold window) all pass. We also ran backend, frontend and end-to-end test cases against the app, and all of them pass.
6. **Next steps.** Password login for staff, email reminders, online fine payment and barcode scanning.

Numbers worth memorising

What	Value
Roles	3: Librarian, Clerk, Member
Database tables (models)	4: members, books, issues, reservations
Routes	15 in <code>app.py</code>
Templates (pages)	10 HTML files, all extending <code>base.html</code>
Loan limits (books / days)	UG 2/30 · PG 4/30 · RS 6/90 · FA 10/180
Late fine	Rs. 2 per day (<code>PENALTY_RATE = 2.0</code>)
Reservation hold	7 days
Unused-book report window	5 years (365 × 5 days)
Reservation states	Waiting, Hold, Fulfilled, Expired
Unit tests	3, all pass (<code>3 passed in 0.24s</code>)

3. Who presents what

This is a suggested split. It gives everyone a similar amount of speaking and keeps each person on one topic. Change it if your team has already agreed on something else. For a 15-minute slot, aim for about 2.5 minutes per person.

Speaker	Slides	Topic
Aryan Raj	1–4	Introduction, the problem, how the system works, the three roles
Ayush Ranjan	5–8	Functional requirements, business rules, tech stack, architecture
Banoth Vijender	9–11	Data flow diagrams: level 0 and both level 1 diagrams
Bathina Shajith	12–16	UML: use case, class, sequence, activity and state diagrams
Battu Manjula	17–20	API reference, pages, screenshots and the live demo
Bhanu Praharsha Kasaraneni	21–25	Unit, backend, frontend and integration tests, future scope, closing

Handover lines

A one-line handover between speakers makes the talk feel rehearsed. For example: "That's what the system has to do. Ayush will show how we built it." Keep each handover to one sentence.

Everyone should be able to answer

- What the project does, in one sentence.
- The four tables and how they relate (one member or book has many issues and many reservations).
- The loan limits, the Rs. 2 fine and the 7-day hold.
- How to run the app and the tests.

4. Slide by slide

01 Cover: Library Information System

Aryan

On the slide

Project title, "Software Engineering Lab project", the six team members with roll numbers, and an illustration of a student in a library.

Say this

Good morning. We're presenting the Library Information System, a web app that lets a college library issue, return and reserve books, with separate views for the librarian, the clerk and members. I'm Aryan, and with me are Ayush, Banoth, Bathina, Battu and Bhanu.

Likely questions

Who built what?

Answer with your real split of work. If the code was written together, say so, and name who led each part (code, diagrams, testing, slides).

02 A paper register can't answer three daily questions

Aryan

On the slide

Three numbered questions: who has this copy right now, when is it due and what is the fine, and who asked for it first. A dark bar underneath says "LIS answers each one from a single database."

Say this

A library desk answers these three questions all day. With a paper register, finding who has a copy means reading pages by hand. The due date and fine depend on the member's category, so they're worked out manually. And with no queue, a returned book goes back on the shelf instead of to the person waiting for it. Our system answers all three from one database.

Facts to know

- Question 1 is answered by the `issues` table: a row with `return_date` empty means the book is out.
- Question 2 is answered by `due_date` (set at issue from the category) and the fine calculated on return.
- Question 3 is answered by the `reservations` table, ordered by `reservation_date`.

Likely questions

Is this a real library's problem or an assumed one?

It's the standard library circulation problem used in software engineering courses. The rules (limits per category, fines, holds, unused-book report) come from the lab problem statement.

03 How a book moves through the system

Aryan

On the slide

Six steps with arrows: Catalogue (clerk), Issue (clerk), Reserve (member), Return (clerk), Hold (system), Reports (librarian), each with a one-line note.

Say this

Here's the whole system in one line, following a single copy of a book. The clerk adds it to the catalogue with its ISBN, rack and number of copies. The clerk issues it after the system checks the member's limit and any holds. While it's out, a member can reserve it and join the queue. When it comes back, the system charges Rs. 2 per late day and holds the copy for the first person in the queue for 7 days. On top of all this, the librarian gets reports on overdue books and books nobody has borrowed in five years.

Facts to know

- Catalogue: `POST /books/add` sets `available_copies = total_copies`.
- Issue: `POST /issue`. Return: `POST /return`. Reserve: `POST /reserve`.
- Hold expiry runs in `check_reservations()`, a `before_request` hook that runs before every request.

Likely questions

Why is Hold marked "System" and not a person?

Nobody triggers it by hand. The return route puts the reservation on hold, and the `before_request` hook expires it after 7 days.

04 Three roles, three views

Aryan

On the slide

Three cards. Librarian: manages members, prints overdue reminders, checks unused books. Clerk: adds and removes books, issues books, takes returns and fines. Member: searches the catalogue, reserves books.

Say this

There are three roles, and each sees a different dashboard. The librarian runs the library: members and reports. The clerk works the counter: the catalogue, issues and returns. Members search and reserve. The server checks the role on every protected route, so a member can't open a clerk page even by typing its URL.

Facts to know

- Login is at `POST /login`. Librarian and Clerk choose their role from a dropdown. A member types a member code (for example U001), which must exist in the `members` table.
- The role is stored in the Flask session (`session['role']`). Protected routes check it and return HTTP 403 "Unauthorized" for the wrong role.
- `dashboard.html` is one template that shows a different menu for each role.

Likely questions

How is access control done?

Role-based access control. After login, the role is kept in the session, and each protected route checks it first, for example `if session.get('role') != 'Clerk': return "Unauthorized", 403`.

Why doesn't the clerk have a password?

For the lab version, login selects a role so the demo is quick. Password login with hashed passwords is the first item in future scope. See section 8.

05 Functional requirements

Ayush

On the slide

A table of ten requirements, FR1 to FR10, each with its actor.

Say this

These are the ten things the system must do. Each one maps to one route in the code, which made them easy to build and test. The core is FR6 and FR7, issuing within the category limit and returning with a fine. The others follow the same pattern.

Facts to know

ID	Requirement	Route
FR1	Sign in by role or member code	POST /login
FR2	Add a book	POST /books/add
FR3	Remove a book with no copy on loan	GET /books/delete/<isbn>
FR4	Search by ISBN, title or author	GET /search?q=
FR5	Register and remove members	/members , /members/delete/<code>
FR6	Issue within the category limit	POST /issue
FR7	Return and charge any fine	POST /return
FR8	Reserve a book with no free copy	POST /reserve
FR9	Print overdue reminders	GET /reports/reminders
FR10	List books unused for five years	GET /reports/unused

Likely questions

What are the non-functional requirements?

Usability (simple role-based menus, printable documents), security (role checks on the server, parameterised SQL through the ORM), maintainability (rules kept as constants, one route per requirement), portability (one Python process and one SQLite file), and testability (pytest suite).

What's the difference between functional and non-functional requirements?

A functional requirement is something the system does (issue a book). A non-functional requirement is a quality of how it does it (fast, secure, easy to use).

06 Business rules

Ayush

On the slide

Left table: loan limits by category. Right table: late fine Rs. 2/day, reservation hold 7 days, unused report 5 years.

Say this

These are the rules the system enforces. The number of books and the loan period depend on the member's category, from 2 books for 30 days for undergraduates up to 10 books for 180 days for faculty. A late book costs Rs. 2 per day. A reserved book is held for 7 days, and a book with no issue in 5 years shows up on the unused report. All of these are constants in the code, so changing a policy is a one-line edit.

Facts to know

```
PENALTY_RATE = 2.0 # Rs. 2 per day
```

```
CATEGORY_LIMITS = {  
    'UG': {'books': 2, 'days': 30},  
    'PG': {'books': 4, 'days': 30},  
    'RS': {'books': 6, 'days': 90},  
    'FA': {'books': 10, 'days': 180}  
}
```

- Due date = issue date + `days` for the member's category.
- Fine = (return date – due date) in days × 2, only if positive. Returning on the due date costs nothing.
- The unused report uses `today – 365 × 5` days as the cut-off.

Likely questions

What if a member returns a book 5 days late?

5 × Rs. 2 = Rs. 10. The seed data has exactly this case: P001's copy of Design Patterns is 5 days overdue.

Does the fine keep growing if the book isn't returned?

The fine is calculated at the moment of return. Before that, the overdue reminder report lists the loan so the librarian can chase it.

07 Tech stack

Ayush

On the slide

Python 3.11, Flask 3.0.0, Flask-SQLAlchemy 3.1.1, SQLite, Jinja2, Bootstrap 5.3.0, pytest 7.4.3, Git and GitHub.

Say this

We kept the stack small. Python and Flask for the server, SQLAlchemy to map our classes to database tables, and SQLite so the whole database is one file with nothing to install. Pages are Jinja2 templates styled with Bootstrap, and we test with pytest.

Likely questions

Why Flask and not Django?

Flask is small and explicit: each route is a plain function, which made the business rules easy to read and to map to requirements. Django brings an admin panel, its own ORM and more structure, which is more than a four-table app needs.

Why SQLite?

It needs no server and stores everything in one file (`lis.db`), so anyone can run the project. Because we use SQLAlchemy, switching to MySQL or PostgreSQL later is mostly a change to the connection string.

What is an ORM?

Object-relational mapping. We write Python classes (Member, Book, Issue, Reservation) and SQLAlchemy creates the tables and turns our method calls into SQL. It also binds values as parameters, which blocks SQL injection.

What does Jinja2 do?

It's Flask's template engine. The server fills HTML templates with data (for example the list of books) and sends finished pages to the browser.

On the slide

Four boxes left to right: Browser (templates/), Flask (app.py), SQLAlchemy (models.py), SQLite (lis.db), joined by HTTP, ORM and SQL arrows.

Say this

The browser sends a form over HTTP. Flask checks the role and applies the rules. SQLAlchemy turns that into SQL, and SQLite stores it in one file. Every page is rendered on the server and sent back as HTML. All the business logic is in app.py.

Facts to know

- This is a layered (three-tier) architecture: presentation, application logic, data.
- It loosely follows MVC. The models are in `models.py`, the views are the Jinja2 templates, and the Flask route functions act as controllers.
- The `before_request` hook runs before every request and expires lapsed holds, so there's no separate scheduler.

Likely questions

Is this MVC?

Yes, in the Flask sense. Model: SQLAlchemy classes. View: Jinja2 templates. Controller: route functions in `app.py`.

Is it client-server?

Yes. The browser is the client, and the Flask app is the server, with the database behind it.

On the slide

One process circle, "0 Library Information System", with three external entities (Librarian, Clerk, Member) and labelled data flows in and out. Dark labels are inputs, grey labels are outputs.

Say this

The context diagram treats the whole system as one process and shows only what crosses its boundary. The librarian sends member details and report requests and gets back reminders and the unused-book report. The clerk sends book details and issue and return entries and gets back bills, slips and messages. The member sends search terms and reservation requests and gets back results and reservation status.

Facts to know

- Notation: rectangle = external entity, circle = process, arrow = data flow, open-ended box = data store (level 1 only).
- A level 0 DFD has exactly one process and no data stores.

Likely questions

What's the difference between a DFD and a flowchart?

A DFD shows how data moves between processes, stores and outside entities. It has no decisions or order. A flowchart (or activity diagram) shows the order of steps and the decisions.

Why is the database not in the level 0 diagram?

At level 0 the database is inside the system boundary. Data stores appear once we break the system into processes at level 1.

10 DFD level 1: catalogue, search, members, reports

On the slide

Four processes: 1.0 Manage books (clerk), 2.0 Search books (member), 3.0 Manage members (librarian), 4.0 Generate reports (librarian). Data stores D1 Books, D2 Members, D3 Issues. A note says repeated stores are one table.

Say this

Level 1 opens up the system into seven processes; this slide has the four that aren't circulation. Managing books reads and writes the Books store. Search only reads Books. Managing members writes Members and reads Issues, so it can refuse to delete a member with unreturned books. Reports only read: overdue loans and member names for reminders, and the book list with loan history for the unused report.

Facts to know

- D1 = books, D2 = members, D3 = issues, D4 = reservations.
- A double-headed arrow means the process both reads and writes that store.
- Some stores are drawn twice so that lines don't cross; this is allowed in DFD notation.
- Any role can search. The member is shown because the context diagram shows the member's search flow.

Likely questions

What is DFD balancing?

Every flow into or out of the level 0 process must appear in the level 1 diagram. Our level 0 flows (member details, reports, book details, issue and return entries, search, reservations) all appear across slides 10 and 11.

11 DFD level 1: issue, return, reserve

Banoth

On the slide

Processes 5.0 Issue book, 6.0 Return book (clerk), 7.0 Reserve (member), each with its store flows.

Say this

These are the circulation processes. Issue reads the member's category, counts their open loans and checks for a hold, then writes a new issue and takes a copy off the shelf. Return writes the return date and fine, and either moves the oldest waiting reservation to Hold or puts the copy back. Reserve checks there's no free copy and no duplicate, then writes a Waiting reservation.

Facts to know

- 5.0 Issue: reads D2 (category), reads/writes D3 (count open loans, insert issue), reads/writes D4 (hold check; marks the member's own hold Fulfilled), writes D1 (copies – 1).
- 6.0 Return: reads/writes D3 (return date, fine), reads/writes D4 (Waiting → Hold), writes D1 (copies + 1 only when nobody is waiting).
- 7.0 Reserve: reads D1 (free copies), reads/writes D4 (duplicate check, new Waiting row).
- Hold expiry is not drawn as a separate process. It runs inside the request hook and updates D4 and D1 the same way 6.0 does.

Likely questions

Why isn't hold expiry shown?

It isn't triggered by an external entity; it runs automatically before each request. It's shown in the state diagram (slide 16) as the Hold → Expired transition.

12 Use case diagram

Bathina

On the slide

Three actors and ten use cases inside the system boundary. Librarian: Manage members, Print reminders, Unused-book report. Clerk: Manage books, Issue book, Return book. Member: Search catalogue, Reserve book. Return book «include» Calculate fine; Print return slip «extend» Return book.

Say this

This shows who does what. Each actor is linked to the use cases they can start. Return book always includes Calculate fine, because the fine is worked out on every return. Printing the slip is an extension: it only happens when there's a fine or a reservation waiting.

Facts to know

- «include»: the base use case always runs the included one. The arrow points from base to included (Return book → Calculate fine).
- «extend»: optional behaviour under a condition. The arrow points from the extension to the base (Print return slip → Return book).

- Search is open to every role; it's drawn against the member to keep lines readable. Login is common to all actors and left out for the same reason.

Likely questions

What's the difference between include and extend?

Include is mandatory and always happens (every return calculates a fine, even if it's zero). Extend is optional and conditional (the slip prints only when there's a fine or a hold).

Why isn't Login a use case?

It's a precondition shared by every actor, not a goal in itself. Many textbooks leave it out to keep the diagram readable. It could be added as a use case linked to all three actors.

13 Class diagram

Bathina

On the slide

Four classes with every attribute and type: Member, Book, Issue, Reservation. Member 1 — 0..* Issue ("borrows"), Book 1 — 0..* Issue, Member 1 — 0..* Reservation ("reserves"), Book 1 — 0..* Reservation. {PK}, {FK} and [0..1] are marked.

Say this

These are our four classes, exactly as they are in models.py. Issue and Reservation link a member to a book. One member can have many issues and many reservations, and one book can appear in many of each. The book keeps both total and available copies, so we can check stock without counting rows.

Facts to know

Class	Attributes
Member	code (PK), name, category (UG/PG/RS/FA), join_date
Book	isbn (PK), title, author, publisher [0..1], rack_no [0..1], total_copies, available_copies, date_added
Issue	id (PK), book_isbn (FK), member_code (FK), issue_date, due_date, return_date [0..1], penalty_paid
Reservation	id (PK), book_isbn (FK), member_code (FK), reservation_date, status, hold_until [0..1]

- An open loan is an Issue with `return_date` empty.
- Relationships are defined with `db.relationship(..., backref=...)`, so `issue.member` and `member.issues` both work.

Likely questions

Why are there no methods in the classes?

They are entity classes that hold data. The behaviour (issuing, returning, reserving) lives in the route functions in `app.py`. A later refactor could move rules like `calculate_fine()` into the model.

Is the database normalised?

Yes, to third normal form. Each table has a single-column primary key, every attribute depends only on that key, and member and book details aren't repeated in Issue or Reservation; they're referenced by foreign key.

Why store available_copies instead of counting?

It's a deliberate denormalised counter for speed and simplicity: the issue route checks one number instead of counting open issues and holds.

14 Sequence diagram: issuing a book

Bathina

On the slide

Lifelines: Clerk, issues.html, app /issue, SQLAlchemy. Messages 1–6 with an alt fragment: [limit ok, no other hold, copy free] saves the issue; [else] flashes the reason refused.

Say this

This follows the issue route step by step. The clerk enters a member code and ISBN, the page posts to /issue, and the route loads the member and the book. It counts the member's open loans and looks for a hold. If the member is within the limit, nobody else holds the book and a copy is free, it saves the issue with the due date and updates stock. Otherwise it shows the reason.

Facts to know

1. Unknown member or ISBN → "Invalid Member Code or ISBN".
2. Open loans $\geq$ limit → "Member has reached their issue limit (N books)."
3. Held for another member → "Book is reserved by another member until DATE."
4. No copy free and no hold → "No copies available for issue."
5. Success: if the hold is this member's, the reservation becomes Fulfilled (the held copy is already off the shelf). Otherwise `available_copies` drops by 1. The Issue gets `due_date = today + days`.

Likely questions

What do the dashed arrows mean?

Return messages: data coming back from the database. Solid arrows are calls.

What is the thin rectangle on the app lifeline?

An activation bar. It shows the period during which the route is running.

What is "alt"?

A combined fragment that works like if/else. Only one of its sections runs, chosen by the guard in square brackets.

15 Activity diagram: returning a book

Bathina

On the slide

Start → Enter member code and ISBN → Issue found? (no → Show error → end) → Record today as return date → Overdue? (yes → Fine = Rs. $2 \times$ days late) → merge → Reserved? (yes → Hold oldest reservation for 7 days; no → Add copy back to stock) → merge → Print slip, or confirm return → end.

Say this

Read the top row left to right, then the bottom row right to left. If there's no open issue, we stop with an error. Otherwise we record the return date and add a fine if the book is late. Then we check the queue: if someone's waiting, the copy goes on a 7-day hold for them; if not, it goes back on the shelf. Finally, a slip prints if there's a fine or a hold; otherwise the clerk sees a confirmation.

Facts to know

- The small diamonds are merge nodes. They join two alternative paths back into one. The large diamonds are decisions.
- There are two final nodes: one for the error path and one for a completed return. That's valid UML.

Likely questions

What's the difference between a merge node and a join node?

A merge (diamond) takes whichever one path arrives. A join (thick bar) waits for all parallel paths. Our paths are alternatives, not parallel, so merge is correct.

Activity diagram vs sequence diagram?

The activity diagram shows the steps and decisions of one workflow. The sequence diagram shows which objects send which messages, in time order.

16 State diagram: reservation lifecycle

Bathina

On the slide

States Waiting → Hold → Fulfilled, and Hold → Expired. Transitions: reserve [no copy free]; copy returned [next in line] / hold 7 days; issued to this member; [today > hold_until]. A note says that when a hold lapses, the next waiting reservation moves to Hold, or the copy goes back to stock.

Say this

A reservation's status field moves through four values. It starts as Waiting because you can only reserve a book with no free copy. When a copy comes back, the oldest waiting reservation moves to Hold for seven days. If the member collects it, it becomes Fulfilled. If not, it becomes Expired and the copy passes to the next person, or back to the shelf.

Facts to know

- Transition labels use the format `event [guard] / action`.
- Waiting → Hold happens in `/return` or when the previous hold expires. Hold → Fulfilled happens in `/issue`. Hold → Expired happens in the `before_request` hook.
- Fulfilled and Expired are final states.

Likely questions

Why a state diagram for Reservation and not for Book?

Reservation has the most interesting life: four named states stored in its status column, with clear events and guards. A book's state is just a count of copies.

On the slide

Method, endpoint, role and purpose for `/`, `/login`, `/logout`, `/dashboard`, `/books`, `/books/add`, `/books/delete/<isbn>` and `/search`.

Say this

These are our routes for sign-in and the catalogue. Every response is either an HTML page or a redirect with a message. The login route takes a role and, for members, a member code. Adding a book takes six fields.

Facts to know

- `POST /login` form fields: `role`, `user_id`. An invalid member code shows "Invalid Member Code".
- `POST /books/add` form fields: `isbn`, `title`, `author`, `publisher`, `rack_no`, `total_copies`. A duplicate ISBN is refused.
- `/search?q=` matches the text against ISBN, title and author.

Likely questions

Is this a REST API?

No. It's a server-rendered web app, so the routes return HTML pages, not JSON. A JSON API for a mobile app would be a natural extension.

What's the difference between GET and POST here?

GET loads a page or runs a search. POST submits a form that changes data, such as login, adding a book, issuing and returning.

18 API reference: circulation and reports

On the slide

Routes for `/members`, `/members/delete/<code>`, `/issue`, `/return`, `/reserve`, `/reports/reminders` and `/reports/unused`.

Say this

These are the circulation and report routes. Issue and return both take a member code and an ISBN. Reserve takes an ISBN from the logged-in member. The two report routes are for the librarian only.

Facts to know

- `/issue` and `/return`: GET shows the form, POST processes it. Both are clerk-only.
- `/return` renders `print_return.html` when there's a fine or a reservation slip; otherwise it flashes "Book returned successfully. No penalty."
- `/reserve`: if a copy is free, the member is told there's no need to reserve; a second Waiting reservation for the same book is refused.
- `/reports/reminders`: issues with no return date and a due date before today.
- `/reports/unused`: books with no issue in the last five years.

On the slide

All ten templates with what each one shows.

Say this

Every page extends base.html, which holds the navbar, the messages and the print styles. Two design choices are worth pointing out. The dashboard is one template that shows a different menu per role. And issue and return share one form, so the clerk sees the same layout for both.

Facts to know

- `base.html` uses a print stylesheet: `.no-print` is hidden on paper, `.print-only` appears only on paper.
- `login.html` shows the member code field only when "Member" is selected (a small JavaScript function).
- `book_search.html` shows the Reserve button only when the role is Member and available copies are 0.
- `print_reminder.html` puts one reminder per printed page (`page-break-after: always`).

Likely questions

What is template inheritance?

A child template extends a base template and only fills in its content block, so shared parts like the navbar are written once.

20 The app running on demo data

On the slide

Three screenshots: the clerk's return page with a Rs. 10 penalty bill and a reservation slip, a member's search showing Reserve on an out-of-stock book, and the librarian's dashboard.

LIS Demo Role: Clerk (Clerk) Dashboard Logout

Print Document Back to Returns

PENALTY BILL

Member Code: P001
 Book ISBN: B002 (Design Patterns)
 Due Date: 2026-09-23
 Return Date: 2026-09-28
 Penalty Amount: Rs. 10.0

RESERVATION SLIP

The following reserved book is now available:
 Member Code: R001
 Book ISBN: B002 (Design Patterns)
 Available Until: 2026-10-05
 Please present this slip at the circulation desk within 7 days.

P001 returns B002 five days late: Rs. 10 bill, plus a slip for R001.

LIS Demo Role: Member (R001) Dashboard Logout

Library Catalog

Design Search

Results for "Design"

ISBN	Title	Author	Rack	Available	Action
B002	Design Patterns	GoF	R1	Out of stock	Reserve

R001 searches "Design"; the book is out, so Reserve appears.

LIS Demo Role: Librarian (Librarian) Dashboard Logout

Dashboard - Welcome Librarian

Members
Manage Members

Reports
Overdue Reminders
Unused Books (5 Yrs)

The librarian's dashboard.

Say this

These are screenshots from the running app. The big one is the best example: P001 returns Design Patterns five days late, so the system charges Rs. 10. Because R001 had reserved it, the same screen prints a reservation slip holding the book for seven days.

If you give a live demo, follow the script in section 6.

21 Unit tests: 3 of 3 pass

Bhanu

On the slide

test_due_date_calculation, test_penalty_calculation and test_reservation_hold, all Pass, with the pytest output "3 passed in 0.24s".

Say this

Our unit tests cover the three rules that involve dates and money: the due date, the fine and the hold window. They run with pytest, create their own test rows and clean them up afterwards. All three pass.

Facts to know

- The fixture creates a test client, and after each test deletes the test member (TEST_T001), book (TEST_B001) and their issues and reservations.
- Due date test: a UG issue made today has a due date 30 days out.
- Penalty test: 5 days overdue × Rs. 2 = Rs. 10.
- Hold test: after moving a reservation to Hold, `hold_until` is after today.
- Run `python seed.py` once before `pytest tests/` so the tables exist.

Likely questions

What is a fixture?

A pytest function that sets up what a test needs (here a test client and app context) and cleans up afterwards.

Are these white-box or black-box tests?

The unit tests are white-box: they're written knowing the formulas in the code. The test cases on the next slides are black-box: they check inputs and outputs through the app without looking inside.

22 Backend test cases

Bhanu

On the slide

Nine server-side cases, all Pass: over the limit, no free copy, held for someone else, return on the due date, return five days late, duplicate ISBN, deleting a book on loan, deleting a member with unreturned books, reserving the same book twice.

Say this

We ran each of these through Flask's test client on freshly seeded data. The boundary cases matter most: a UG member with two open loans is refused a third, a return on the exact due date has no fine, and five days late costs exactly Rs. 10. Every refusal comes back with a specific message.

Facts to know

- These are boundary value and equivalence class tests: exactly at the limit (2 loans for UG), exactly on the due date, one duplicate.
- Messages seen: "Member has reached their issue limit (2 books).", "No copies available for issue.", "Book is reserved by another member until ...", "Book returned successfully. No penalty.", "Book with this ISBN already exists", "Cannot delete book. Some copies are currently issued.", "Cannot delete member. They have unreturned books.", "You have already reserved this book."

23 Frontend test cases

Bhanu

On the slide

Eight user-facing cases, all Pass: empty issue form blocked, 0 copies blocked, unknown member code shows an error, member opening a clerk page is denied, delete asks to confirm, Reserve button for an out-of-stock book, printing hides the navbar and buttons, SQL injection text treated as plain text.

Say this

These check what the user sees. Required fields and a minimum of 1 copy stop bad forms in the browser. An unknown member code shows an error. A member who types a clerk URL gets access denied. Delete asks for confirmation. The Reserve button only appears when it makes sense. Printed bills hide the buttons. And typing an SQL injection string into search does nothing harmful, because the ORM binds it as a parameter.

Facts to know

- The empty-form and 0-copies checks come from HTML `required` and `min="1"`, and were tested in a real Chromium browser.
- Access denied is HTTP 403 from the server-side role check.

24 Integration tests: end-to-end scenarios

Bhanu

On the slide

Five scenarios, all Pass: late return while a member waits; another member asks for the held copy; the waiting member collects it; a hold passes its end date; seed data then test suite.

Say this

These scenarios cross every layer: form, route, rules and three tables. P001 returns Design Patterns five days late while R001 is waiting; the system charges Rs. 10, holds the copy for R001 and prints both documents. U002 is refused that copy during the hold. R001 collects it and the reservation becomes Fulfilled. We also moved a hold past its end date, and on the next request the system expired it and moved the next member to Hold.

Likely questions

What's the difference between unit and integration testing?

A unit test checks one piece on its own, such as the fine formula. An integration test checks that the pieces work together, such as a return that updates the issue, the reservation and the book, and prints the right page.

Did you do system or acceptance testing?

The end-to-end scenarios are a form of system testing: they use the whole app as a user would. Acceptance testing would be the librarian trying it against their requirements.

25 Future scope and thank you

Bhanu

On the slide

Password login for staff, email reminders for overdue books, online payment of fines, barcode scanning at the counter. "Thank you". The right half is left for an image.

Say this

To extend the system, we'd add password login for staff, email reminders using the same overdue query as the reminder report, online payment on top of the penalty field we already store, and barcode scanning so the clerk doesn't type ISBNs. Thank you. We're happy to take questions.

5. How the code works

Files

File	What it contains
<code>app.py</code>	The Flask app, configuration, business constants, the <code>before_request</code> hook and all 15 routes (about 315 lines).
<code>models.py</code>	The four SQLAlchemy models and their relationships.
<code>seed.py</code>	Drops and recreates the tables, then loads demo members, books, issues and one reservation.
<code>tests/test_lis.py</code>	Three pytest unit tests and a cleanup fixture.
<code>templates/</code>	Ten Jinja2 templates extending <code>base.html</code> .
<code>requirements.txt</code>	Flask 3.0.0, Flask-SQLAlchemy 3.1.1, pytest 7.4.3.

Issuing a book (`POST /issue`)

1. Only the Clerk role may call it (403 otherwise).
2. Load the member and the book. If either is missing: "Invalid Member Code or ISBN".
3. Count the member's open issues (`return_date` is empty). If the count is at the category limit: refused.
4. Look for a reservation on this book with status Hold. If it belongs to another member: refused with the hold date.
5. If no copy is free and there's no hold: "No copies available".
6. If the hold belongs to this member, mark it Fulfilled (the copy was already set aside). Otherwise reduce `available_copies` by 1.
7. Create the Issue with `due_date = today + days` for the category, commit, and show "Book issued successfully."

Returning a book (`POST /return`)

1. Clerk only. Find the open issue for this member and ISBN, or show "No active issue found".
2. Set `return_date` to today. Overdue days = return date – due date. If positive, fine = days × 2, saved in `penalty_paid`.
3. Find the oldest Waiting reservation for the book. If there is one, set it to Hold with `hold_until = today + 7`. If not, add 1 to `available_copies`.
4. If there's a fine or a hold, show the printable bill and/or slip. Otherwise show "No penalty".

Reserving (`POST /reserve`)

1. Member only. If the book has a free copy: "Book is currently available, no need to reserve."
2. If this member already has a Waiting reservation for it: "You have already reserved this book."
3. Otherwise add a Waiting reservation dated today.

Hold expiry (`before_request`)

Before every request, `check_reservations()` finds Hold reservations whose `hold_until` is before today and marks them Expired. For each one, the oldest Waiting reservation for that book becomes Hold for 7 days. If nobody is waiting, the copy goes back to `available_copies`.

Reports

- **Overdue reminders:** all issues with no return date and a due date before today, printed one per page.
- **Unused books:** collect the ISBNs issued in the last 5 years, then list every book not in that set.

Search

`GET /search?q=text` returns books whose ISBN, title or author contains the text. The query goes through SQLAlchemy, so the text is sent as a parameter, not pasted into SQL.

6. Running the app and the live demo

Setup

```
git clone https://github.com/BhanuPraharsa/SE_lab
cd SE_lab
pip install -r requirements.txt
python seed.py          # creates lis.db with demo data
python app.py           # open http://127.0.0.1:5000
pytest tests/           # 3 passed
```

Run `python seed.py` again right before the demo. It resets the data, and the overdue example depends on today's date.

Bootstrap loads from the internet. If the lab has no internet, the pages still work but look unstyled, so check the connection before you start.

Demo logins

Login as	Code	Who
Librarian	none	Choose "Librarian" in the dropdown
Clerk	none	Choose "Clerk" in the dropdown
Member	U001	Alice Smith, UG, already has 1 book (B001)
Member	P001	Bob Jones, PG, has B002 five days overdue
Member	R001	Charlie Brown, RS, waiting for B002
Member	F001	Dr. David, faculty
Member	U002	Eve Davis, UG, no loans

Seed books

ISBN	Title	Copies	Why it's there
B001	Software Engineering (Sommerville)	2, 1 free	One copy on loan to U001
B002	Design Patterns (GoF)	1, 0 free	Overdue with P001; R001 waiting
B003	Clean Code (Robert Martin)	3, 3 free	Free to issue
B004	Old Physics (Newton)	1, 1 free	Not issued in 5 years: appears on the unused report

Demo script (about 3 minutes)

- 1. Member view.** Log in as R001. Search "Design". Design Patterns shows "Out of stock" and a Reserve button. (R001 has already reserved it in the seed data, so don't click Reserve again.) Log out.

2. **Limit check.** Log in as Clerk. Issue B003 to U001: succeeds (U001 now has 2). Issue B003 to U001 again:
"Member has reached their issue limit (2 books)."
3. **The key moment.** Return B002 for P001. The page shows a Rs. 10 penalty bill and a reservation slip for R001, valid for 7 days. Click Print Document to show the navbar disappear.
4. **Hold protection.** Try to issue B002 to U002: "Book is reserved by another member until ...". Then issue B002 to R001: succeeds.
5. **Librarian.** Log in as Librarian. Open Overdue Reminders, then Unused Books (Old Physics is listed).

Stick to this script in the demo. Section 8 explains which unusual inputs to avoid.

7. Question bank

About the project

What happens if two members reserve the same book?

Both are stored as Waiting. The one with the earlier `reservation_date` gets the hold first. When their hold is fulfilled or expires, the next one moves to Hold.

What if the reserved member never collects the book?

After 7 days the hold expires on the next request. The book moves to the next waiting member, or back to the shelf.

Can a member reserve a book that's on the shelf?

No. The system says it's available and there's no need to reserve.

Can a book with copies on loan be deleted?

No. If `total_copies` differs from `available_copies`, deletion is refused.

Can a member with unreturned books be deleted?

No. The route counts open issues first and refuses.

How is the due date calculated?

Issue date plus the category's loan days: 30 for UG and PG, 90 for RS, 180 for faculty.

How does the system know a book is overdue?

An issue is overdue when `return_date` is empty and `due_date` is before today. That's the reminder report's query.

Where is the fine stored?

In `Issue.penalty_paid`, set at return time.

Why is the session secure enough for a lab?

Flask signs the session cookie with the secret key, so a user can't change their role by editing the cookie.

How would you scale this for a real university?

Move to PostgreSQL, add real authentication, run under a production server such as Gunicorn, add indexes on the foreign keys, and run hold expiry as a scheduled job.

What was the hardest part?

The reservation queue: making sure a returned book goes to the right member, that a hold blocks everyone else, and that an expired hold passes the book on automatically.

Software engineering theory

Which SDLC model did you follow?

Answer with what your team actually did. For a lab project built requirement by requirement, "iterative and incremental" is usually accurate: gather requirements, design with UML, build a feature, test it, repeat.

What is UML? Which diagrams are structural and which behavioural?

Unified Modelling Language, a standard notation for software design. Structural: class diagram (and component or deployment diagrams). Behavioural: use case, sequence, activity and state diagrams.

What is an actor?

Anyone or anything outside the system that interacts with it. Ours are Librarian, Clerk and Member.

What is multiplicity?

How many objects can be linked in an association. "1" to "0..*" means one member can have zero or more issues, and each issue belongs to exactly one member.

Association, aggregation, composition?

Association is any link. Aggregation is a whole-part link where parts can exist alone. Composition is a whole-part link where parts die with the whole. Ours are plain associations: an issue refers to a member and a book, but isn't a part of either.

Cohesion and coupling?

Cohesion is how focused one module is; coupling is how much modules depend on each other. We aim for high cohesion and low coupling: models only describe data, templates only display, and the database can change behind SQLAlchemy.

What is a data dictionary?

A list of every data item with its type and meaning. Our class diagram (slide 13) works as one.

What are the levels of testing?

Unit (one function), integration (modules together), system (whole app) and acceptance (by the user). We did unit, integration and system-level test cases.

White-box vs black-box testing?

White-box tests are written from the code's internals. Black-box tests only use inputs and expected outputs. Our unit tests are white-box; our backend and frontend cases are black-box.

What is boundary value analysis?

Testing at the edges of a valid range, where bugs cluster. Examples: a UG member at exactly 2 loans, a return exactly on the due date.

What is regression testing?

Re-running existing tests after a change to make sure nothing broke. Running `pytest tests/` after every change does this.

What is SQL injection and how is it prevented?

An attack where input is treated as SQL code. SQLAlchemy sends inputs as bound parameters, so the text is never run as SQL. We tested this with `' OR 1=1 --`.

What is a foreign key?

A column that refers to another table's primary key. `Issue.book_isbn` refers to `Book.isbn`, and `Issue.member_code` refers to `Member.code`.

What is feasibility analysis?

Checking whether a project is worth doing: technical (can we build it with Flask and SQLite, yes), economic (all tools are free and open source) and operational (the staff can use a simple web page).

8. Known weak spots (team only)

FOR THE TEAM. DON'T PRESENT THIS.

We found these while testing. They're normal for a lab prototype and aren't on the slides. If an examiner finds one, don't argue; say it's a known gap and give the fix. That comes across better than being surprised. Better still, fix them in the code before the presentation. Most are one or two lines.

What happens	If asked, say	Fix
Typing "abc" or leaving copies empty in a hand-made request gives an error page (500). The form itself blocks this in the browser.	"The browser validates it; we'd add server-side validation too."	Wrap <code>int(...)</code> in try/except; reject values below 1.
A request with 0 or negative copies, sent without the form, is saved.	Same as above.	Check <code>copies >= 1</code> in <code>add_book</code> .
Deleting or reserving an ISBN or member code that doesn't exist (typed into the URL) gives a 500 error.	"We'd return 404 for unknown records."	Use <code>Book.query.get_or_404(isbn)</code> .
A book that has ever been borrowed can't be deleted (500), because SQLAlchemy tries to blank the issue's foreign key.	"History is kept on purpose; we'd mark books withdrawn instead of deleting."	Add a cascade rule, or a soft-delete flag.
Librarian and Clerk log in without a password; <code>SECRET_KEY</code> is written in <code>app.py</code> .	"It's first on our future scope list."	Flask-Login with hashed passwords; read the key from an environment variable.
<code>/books</code> and <code>/search</code> open without logging in; the add form shows, but adding is still blocked.	"The catalogue is public by design; changes need the Clerk role."	Check the role in <code>list_books</code> .
Searching "%" returns every book.	"It's a wildcard in LIKE; harmless."	Escape % and _ in the query.
A member category outside UG/PG/RS/FA, sent without the form, is saved and gets a limit of 0.	"The dropdown only offers valid ones; we'd check on the server too."	Reject unknown categories.
A member can reserve a book they already have on loan.	"We'd add that check."	Refuse if the member has an open issue for that ISBN.
A book added recently and never issued appears on the 5-year unused report.	"We'd only count books older than five years."	Add <code>Book.date_added <= five_years_ago</code> to the query.
The printed reminder shows an empty date line.	"A missing template variable; one-line fix."	Pass <code>current_date=today</code> to <code>render_template</code> .
The app uses <code>utcnow()</code> and the seed uses <code>now()</code> ; between midnight and 5:30 a.m. in India the dates differ by a day. <code>join_date</code> 's default is fixed when the server starts.	"We'd use one clock everywhere."	Use <code>date.today()</code> consistently; pass a function as the default.
Tests use the real <code>lis.db</code> . Without <code>seed.py</code> first, 2 fail and 3 error.	"We'd point tests at a separate in-memory database."	Set <code>SQLALCHEMY_DATABASE_URI='sqlite://'</code> and <code>create_all()</code> in the fixture.

Two clerks issuing the last copy at the same instant could both succeed.

"Only one counter in a lab setup; a real deployment would lock the row."

Conditional update on `available_copies > 0`.

None of these affect the demo script in section 6.

9. Glossary

Term	Meaning in this project
Issue	One loan of one book to one member. Open until <code>return_date</code> is set.
Reservation	A member's place in the queue for a book with no free copy.
Hold	A returned copy set aside for the first member in the queue, for 7 days.
Category	UG, PG, RS or FA; sets the loan limit and period.
Route	A URL plus a Python function in <code>app.py</code> that handles it.
Session	A signed cookie that remembers the logged-in role and user.
Flash message	A one-time message shown at the top of the next page ("Book issued successfully.").
ORM	Object-relational mapper; SQLAlchemy turns Python classes into tables and calls into SQL.
<code>before_request</code>	A Flask hook that runs before every request; ours expires lapsed holds.
HTTP 403	"Forbidden": returned when a logged-in user has the wrong role for a route.
Seed data	Demo records loaded by <code>seed.py</code> .
Fixture	A pytest helper that prepares and cleans up test data.
DFD	Data flow diagram: processes, data stores, external entities and data flows.
«include» / «extend»	Use case relationships: always part of / optionally added to another use case.
Merge node	Small diamond in an activity diagram where alternative paths rejoin.
Guard	A condition in square brackets that must be true for a transition or branch.